

QUES1:

Which one is fast between list and dictionary in creation, insertion, searching, deletion?

ANS:

Time Complexity of List vs Dictionary

Operation	List	Dictionary
Creation	$O(n)$	$O(n)$
Insertion	$O(1)$ (append)	$O(1)$ average
Searching	$O(n)$	$O(1)$ average
Deletion	$O(n)$	$O(1)$ average
Traversing	$O(n)$	$O(n)$

INSERTION:

Data Structure	Time Complexity
List (append)	$O(1)$ average
List (insert at beginning/middle)	$O(n)$
Dictionary	$O(n)$ average

List Insertion

1. Append at End

```
number = [1, 2, 3]
```

```
number.append(4)
```

Python adds element at the end.

|1|2|3|4|

So average complexity:

$O(1)$

2. Insert in Middle/Beginning

`number.insert(0, 100)`

All elements shift right:

Before: |1|2|3|

After: |100|1|2|3|

Shifting takes time:

$O(n)$

Dictionary Insertion

`data["a"] = 10`

Python:

1. Calculates hash
2. Directly stores value

No shifting needed.

Average complexity:

$O(1)$

CREATION:

In **creation**, usually **List is faster** than Dictionary.

Why?

List Creation

```
number = [1, 2, 3]
```

Python only stores values sequentially in memory.

|1|2|3|

Very simple process.

Dictionary Creation

```
data = {"a":1, "b":2}
```

Python must:

1. Create a hash table
2. Calculate hash for each key
3. Store key-value pairs

Extra internal work is required.

SEARCHING:

List

A list works like:

```
[10, 20, 30, 40]
```

Stored in memory continuously:

|10|20|30|40|

To search 30, Python may check:

- 10 ❌
- 20 ❌
- 30 ✅

So searching takes more time.

Dictionary

A dictionary works using a **hash table**.

```
data = {"a":10, "b":20}
```

Python converts the key into a hash value:

`hash("a")` → memory location

So it directly jumps to the value instead of checking one by one.

NOTE:

If the required element is at the **first position** in both a list and a dictionary, then usually the **list can be slightly faster** for that one specific case.

Reason:

- **List:**
 - Python checks first element directly.
 - No hashing calculation needed.
- **Dictionary:**
 - Python first calculates the hash of the key.
 - Then finds the memory location.

DELETION:

List Deletion

Example:

```
number = [10, 20, 30, 40]
```

```
del number [1]
```

Delete 20

Before:

|10|20|30|40|

After:

|10|30|40|

Python shifts:

- 30 → left
- 40 → left

Shifting takes time.

Complexity

$O(n)$

Dictionary Deletion

Example:

```
data = {"a":10, "b":20}  
del data["b"]
```

Python:

1. Computes hash of "b"
2. Directly removes entry

No shifting required.

Complexity

$O(1)$

Dictionary is faster because dictionary deletes directly using hashing, while list may need shifting of elements.

QUES2:

Which searching algorithm is used for list and dictionary in python.?

ANS:

Python list mainly uses **Linear Search**.

Example:

```
number = [10, 20, 30, 40]
```

Searching 30:

10 → 20 → 30

Python checks elements one by one.

Complexity

O(n)

Searching in Python Dictionary

Python dictionary uses a **Hash Table** with **Hashing**.

Example:

```
data = {"a":10, "b":20}
```

Searching:

```
data["b"]
```

Python:

1. Computes hash of "b"
2. Directly jumps to memory location

Complexity

O(1) average

QUES3:

Which algorithm is used for insertion of element in dictionary before python 3.6 and after python 3.6?

ANS:

Before Python 3.6

Python dictionary used:

Hash Table with sparse array storage

Characteristics

- Order was **not guaranteed**
- Elements could appear in random order
- More memory used

Insertion Process

1. Compute hash of key
2. Find bucket/index
3. Store key-value pair

Algorithm used:

Hashing + Open Addressing

Collision handling mainly used probing techniques.

After Python 3.6

Python introduced:

Compact Dictionary

(Officially guaranteed ordered from Python 3.7)

Characteristics

- Maintains insertion order

- Uses less memory
- Faster iteration

Internal Working

Python now stores:

1. Separate compact array for entries
2. Hash table storing indexes to entries

Still uses:

Hash Table + Hashing

But memory layout became optimized.